Redes neuronales

- Se basa en la idea de neuronas conectadas entre sí mediante un peso.
    - El peso de la conexión entre dos neuronas tiende a aumentar cuando se activan simultáneamente (regla de Hebb).
- Formadas por una capa de entrada con J neuronas (número de características generalmente), H capas ocultas con P neuronas cada capa y una capa de salida con O neuronas (número de clases o valor predicho).
- Cada neurona (sin contar las de entrada) tiene asociado unos pesos a las entradas que reciben, W.
- Las funciones de activación de las neuronas pueden ser: lineal, sigmoide, softmax, ReLU, tanh.
    - En capas internas se usa principalmente ReLU,
    - En capas de salida se usa:
        - Lineal para regresión. Si se quiere solo salidas positivas ReLU.
        - Sigmoide o logística. Clasificación binaria y clasificación multisalida (más de una etiqueta predicha) sin exclusividad.
        - Softmax. Cuando las clases son excluyentes entre sí, ya sea con una sola salida o varias.
- Complejidad del entrenamiento: $O(i \cdot n \cdot (m \cdot J + (H-1) \cdot H^2 + H \cdot O))$
- Hiperperámetros: número de capas ocultas, número de neuronas, función de activación en las capas ocultas y de salida, ratio de aprendizaje
- Descenso del gradiente para entrenar las redes.
    - Es el algoritmo usado para el entrenamiento de las redes neuronales. Consiste en recalcular los pesos que conforman las conexiones entre las neuronas de la red usando el error obtenido entre la salida y el valor esperado, pasando este error desde la capa de salida hasta la capa de entrada.
    - Para medir que tan mal está prediciendo el modelo se usa una función de pérdida o costo. En regresión se suele usar el MSE o el MAE, según se quiera que los valores atípicos tengan más o menos importancia. En clasificación se usa la entropía cruzada.
    - El objetivo del algoritmo es minimizar esta función de pérdida. El gradiente de la función es la derivada parcial de dicha función en función de cada una de las características de los datos. La evaluación de una instancia en el gradiente indicará haciendo donde debe moverse para que la función aumente, por lo que el algoritmo se mueve en dirección contraria, haciendo que el gradiente disminuya hasta un mínimo. Es posible que existan más de un mínimo: muchos locales y uno global.

- Este algoritmo no asegura siempre la convergencia, aunque generalmente sí.
- Los hiperparámetros sería:
  - tasa de aprendizaje: en función del valor tardará más (valor muy pequeño) o menos (valor grande). Si es demasiado grande es posible que no converga.
  - Número de iteraciones para la búsqueda: cuantas veces se va a realizar el algoritmo para buscar el mínimo. Un número demasiado pequeño puede provocar que no se llegue al mínimo.
  - Tamaño del lote en caso de que la implementación lo requiera.
- Los principales problemas son varios:
  - Los mínimos locales
  - Los puntos sillas. No es un mínimo, pero la derivada es cero (parte superior de una silla de montar).
  - Gradientes muy grandes, se disparan los pesos, o gradientes muy pequeños y el algoritmo deja de aprender (estacamiento).
- Hay diferentes implementaciones:
  - Por lote: se usan todos los datos para la actualización de los pesos. Es muy lento en grandes datasets, aunque es estable y converge suavemente.
  - Estocástico: se usa solo una instancia para calcular las actualizaciones. Es rapidísimo, pero puede saltarse algún mínimo o incluso oscilar mucho y tardar muchísimo en llegar al mínimo (incluso pudiendo no llegar).
  - Mini-batch: usando un pequeño subconjunto de los datos para actualizar los datos. Es un equilibrio entre los dos anteriores, pero requiere seleccionar correctamente el tamaño del lote.

Algoritmo de clasificación

- **Naive Bayes**.
    - Aprendizaje supervisado.
    - Aprendizaje basado en modelo: aprende las probabilidades de cada clase y las condicionadas de cada característica por clase.
    - Asume que las características de los datos son independientes entre ellas.
    - Buen clasificador, pero mal estimador.
    - Requiere pocos datos para generar las estimaciones.
    - No tiene hiperparámetros que afinar.
    - Complejidad de entrenamiento: O(nm), con n el número de instancias y m las características.
    - Complejidad de inferencia: O(dm), con d el número de clases. Muy rápido.
    - Submodelos según la distribución asumida:
        - **Gaussian Naive Bayes**.
        - **Multinomial Naive Bayes**. Usado en clasificación de textos, ya que es apropiado para características que representan un conteo.
        - **Complement Naive Bayes**. Adaptación del multinomial especialmente bueno para datos desbalanceados. Mejor que MNB para clasificación de textos.
        - **Bernoulli Naive Bayes**. Todas las características de los datos deben ser binarias.

- **K vecinos más cercanos, KNN**.
    - Puede funcionar para clasificación supervisada, clasificación no supervisada.
    - Genera agrupación de los elementos basándose en los K valores más cercanos al elemento.
    - Aprendizaje basado en instancias: NO requiere entrenamiento como tal, pues la decisión se hace en base a los datos de entrenamiento.
    - La adición de datos de entrenamiento puede provocar la generación de nuevas predicciones.
    - MUY sensible a la escala, los datos de menor escala serán subestimadas.
    - Complejidad del entrenamiento: O(1) o O(n), consiste en guardar las instancias.

- Complejidad de la inferencia: O(nm). consiste en calcular la distancia con todas las instancias de entrenamiento, tomar los k más cercanos y asignar la clase mayoritaria.
- A la hora de la asignación teniendo más de dos clases, definir una heurística para determinar en caso de empates.
- Hiperparámetros a afinar: K, distancia.

- **Árboles de decisión.**
  - Aprendizaje supervisado basado en modelo. No paramétrico.
  - **No requiere un procesado de datos**: se pueden soportar hasta valores faltantes.
  - Tiene al **sobreajuste**. Conviene aplicar reducción de características para reducirlo o aplicar restricciones sobre los hiperparámetros: incrementar los que sea min_* y decrementar los que sean max_*.
  - Establece fronteras verticales u horizontales a los ejes de dimensiones.
  - El objetivo es ir reduciendo la incertidumbre de los nodos. Métricas usadas:
    - Impureza de Gini. Una impureza de 0 en un nodo corresponde a una clasificación perfecta (todas las instancias son de la misma clase).
    - Entropía. Suele proporcionar árboles más balanceados.
  - Puede darse que un nodo hijo tenga más impureza que un nodo padre, pero lo ideal es que vaya reduciéndose.
  - La complejidad del entrenamiento es, en el peor caso, $O(n^2m)$, aunque de normal puede ser $O(n \cdot \log(n) \cdot m)$.
  - La complejidad de la inferencia es $O(\log(n))$.
  - Balancear los datos para evitar que el árbol de incline a la clase mayoritaria.
  - Hiperparámetros a afinar: profundidad, mínimo de instancias necesarias para crear una hoja y máximo número de nodos hoja.
  - Algoritmo CART (clasificación y regresión):
    - Dividir el conjunto de entrenamiento seleccionando una característica C y un umbral Tc. Esta elección se realiza intentando obtener la división más pura posible (menor incertidumbre).
    - En función de C y Tc, se crean los dos nodos hijos y se asignan las instancias.
    - Se vuelve a buscar C y Tc en los hijos si no se ha llegado a la máxima profundidad o si no se mejora la impureza.

- **Máquinas de soporte de vectores**.
    - Aprendizaje supervisado.
    - Regresión, clasificación lineal y no lineal, y detección de novedades.
    - Alta efectividad en conjuntos de datos pequeño o medianos, independientemente del número de dimensiones (incluso si m >> n).
    - Le afecta mucho la escala de los datos.
    - No permite sacar las probabilidades por defecto.
    - Usa un subconjunto de los datos (vectores de soporte) para determinar la función de decisión.
    - Funciona determinando los hiperplanos que separen a los conjuntos de elementos dejando el mayor espacio entre ellos.
        - Buscar las instancias más extremas de cada conjunto.
        - Busca cuales son las que generan el mayor espacio entre los conjuntos. A este espacio se le denomina calle. Por la mitad de la calle irá el hiperplano.
            - A cada mitad de la calle se le denomina margen: si puede haber instancias dentro de la calle: soft margin. Sino hard margin. El **hiperparámetro C** define si se opta por un soft margin (valor pequeño) o si se opta por un hard margin (valor alto).
    - Un modelo con sobreajuste tenderá a ser soft margin.
    - Es posible modificar su kernel (función que se le aplicará a los datos para transformar el espacio vectorial) para intentar conseguir una separación lineal (en la transformada) de los datos:
        - Si se trabaja con conjuntos linealmente separables: kernel lineal.
        - Si son polinómicamente separables: kernel polinómico de grado n.
        - Gaussianas: kernel RBF gaussiano.
        - Sigmoide.
    - Complejidad en el entrenamiento: $O(mn^2)$ o $O(mn^3)$. Bastante inviable para conjuntos de muchas instancias.
        - En el caso del LineaSVC, tiene una complejidad de O(mn).
    - Hiperparámetros:
        - **Kernel usado**.
        - **C**. Decide si se opta por un soft margin (valor pequeño) o un hard margin (valor grande).
        - **Gamma**. Coeficiente para los kernel RBF, polinómico y sigmoide. En la RBF decide la estrechez de la campana (a mayor valor más estrecha).

- **Coef0**. Coeficiente independiente en el kernel polinómico y sigmoide.

Algoritmos de clústering

- **K-medias**.
    - Aprendizaje no supervisado.
    - Establece k puntos aleatorios (centroides) y calcula las instancias más cercanas a cada uno de los puntos. A partir de estos puntos se calculan los nuevos centroides, que son las medias.
    - Asume que los datos tienen la forma que se pueda extrapolar de la distancia usada para generar los clústeres.
    - Si los clústeres tienen tamaños diferentes, formas o densidades diferentes, el algoritmo no ofrece buenos resultados.
    - Muy rápido para grandes volúmenes de datos.
    - Se garantiza la convergencia, pero no que la solución sea la óptima. El algoritmo minimiza la inercia (distancia entre las instancias y el centroide más cercano).
    - Complejidad del entrenamiento: O(tknm), con t el número de iteraciones, k el número de centroides.
    - Complejidad de la inferencia: O(km). Para calcular qué centroide es el más cercano.
    - Hiperparámetros: número de centroides (K), distancia usada (para la forma de la distribución de los datos). Posible también el número de ejecuciones para buscar el mínimo global (n_init)
    - La elección del número de centroides se realiza usando la silueta (mínimo dos centroides). Acotado entre -1 y 1. A más cercano a 1 mejor.
    - Derivados:
        - K-medias con inicialización controlada de los centroides.
        - Versión de Elkan. Para conjuntos grandes de datos acelera el entrenamiento, generalmente, reduciendo cálculos innecesarios.
        - Versión de Sculley. Calcula los centroides usando lotes de los datos de entrenamiento.
    - Uso para segmentación de clientes o imágenes, análisis de datos, reducción de dimensionalidad, creación de características, generación de pseudoetiquetas para el aprendizaje semi-supervisado.

- **DBSCAN**.
    - Aprendizaje no supervisado.
    - Define clústeres basados en regiones continuas de alta densidad (igual que Mean Shift) de forma arbitraria.

    - Especializado en detectar valores atípicos.
    - Busca instancias núcleo, que son aquellas que están dentro de una región de alta densidad.
    - Con instancias de alta dimensionalidad pierde sentido.
    - Complejidad de entrenamiento de $O(m^2n)$, no muy adecuado para conjuntos grandes.
    - Hiperparámetros: min_samples (número mínimo de instancias que determinan un vencidario-e), eps (máxima distancia para determinar un vecino) y la métrica.

- **Mezclas gaussianas**.
    - Aprendizaje no supervisado.
    - Versión mayor del k-medias.
    - Soft clustering.
    - Para la detección de anomalías.
    - Modelo generativo.
    - Asume que los datos están en grupos con formas elípticas de diferente tamaño y densidad.
    - Es adecuado eliminar los datos anómalos de los datos antes de realizar el entrenamiento.
    - Se basa en la idea de que k distribuciones gaussianas diferentes (orientación, forma, tamaño).
    - Complejidad del entrenamiento: O(kmn) si se indica que las distribuciones son esféricas o elipsoidales, $O(kmn^2+kn^3)$ en otros casos.
    - La detección de anomalías con estos modelos consiste en comprobar cual es la probabilidad de existencia de instancia según la combinación de distribuciones. Si esa probabilidad es inferior de un determinado umbral (X%), se determina que la instancia es una anomalía.
    - La selección del número de distribuciones se realiza mediante el uso del criterio de información bayesiano (BIC) o del criterio de información de akaike (AIC).
    - Hiperparámetros: número de distribuciones, tipo de la matriz de covarianzas de las distribuciones.

- **Mezclas gaussianas bayesianas**.
    - Aprendizaje no supervisado.
    - Intenta buscar distribuciones de probabilidad para los parámetros de las distribuciones.

    - Determina automáticamente el número de clústeres. Si se especifica un valor, pero realmente se necesitan menos, se descartan los restantes.
    - Hiperparámetros: weight_concentration_prior (un valor bajo hará que el modelo ponga mucho peso en algunos componentes y el resto a cero, un valor alto hará que muchos componentes estén activos en la mezcla. Ayudará a determinar el número de clústeres).

- **Estimación de densidad**.
    - Versión "mejorada" de los histogramas.
    - Estima la función de densidad de la distribución de los datos.
    - No supone ninguna distribución sobre los datos, pero sí usa un tipo de distribución para determinar las montañas.
    - Acepta cualquier número de dimensiones, pero en valores muy altos provoca que el rendimiento baje.
    - Se utiliza principalmente en la representación gráfica de instancias en 1D y 2D.
    - También se puede usar para estimar las distribuciones dentro de otros modelos, como el Naive Bayes.
    - Hiperparámetros: el kernel (especifica la distribución de los datos), y el kernel bandwidth (tamaño del kernel en un punto)

Algoritmos de regresión

- **K vecinos más cercanos, KNN**.
    - Igual que el caso de la clasificación, solo que la salida consiste en calcular la media los K vecinos.
    - Por construcción no asume linealidad en los datos.
    - Hiperparámetros a afinar: K, distancia, fórmula de cálculo de la media.

- **Árbol de decisión**.
    - Igual que en el caso de la clasificación.
    - Los nodos hojas contienen la media de las instancias que contienen.
    - La impureza en este caso vendrá dada por una medida de error o desviación:
        - Error cuadrático medio.
        - Desviación media de Poisson.
- **SVM**.
    - En este caso se opta por modelos con un soft margin máximo.